

MY APPLICATION BOT

GIVE LIKE THIS



Pasted text.txt
Document

THIS IS ABOUT THE APPLICATION STATERGIES DATA



Pasted text(1).txt
Document



Pasted text (2).txt
Document

Phase 22B HealthScore Audit
There are two separate HealthScore concepts — one in the risk/strategy path, one in SOR/venue path. The audit covers both.

1. Strategy HealthScore (0–100)
Generation

File	Function	Line	What it does
strategy_tracker.go	trackerHealthScore()	534–568	Computes 0–100 from WinRate, ProfitFactor, MaxDrawdown, ConsecutiveLosses, TotalTrades



Pasted text(2).txt
Document

OK



Pasted text(3).txt
Document

?



Pasted text(4).txt
Document

?

when i open the url in the mobile no data it is showing trades are zero ?

this is the mobile screen shot , <https://antigravity-git-main-raghavar8088s-projects.vercel.app/mock-trading>

ok

 **mango-db-data.txt**
Document

this as mango db articulture

in the application no trades are displaying give very strong single prompt for this one to paste in the claude agent

 **Pasted text(5).txt**
Document

?

 **Pasted text(6).txt**
Document

ok

 **Pasted text(7).txt**
Document

ok

?

ok

Based on the screenshots, your deployment looks much healthier than the earlier "NO GO" report suggested.

What the screenshots confirm

✓ Engine is running

From:

❏ Bash

```
curl http://localhost/health
```

Response:

❏ JSON

```
{
  "service": "raig-engine-v3",
  "status": "ok",
  "strategies": 606,
  "uptime": "6m34s"
}
```

This confirms:

- Engine booted successfully
- HTTP server is running
- 606 strategies loaded
- No startup panic
- Health endpoint responding

✓ Docker container is running

❏ Bash

```
docker ps
```

shows:

```
antigravity_engine
Up 3 minutes
```

and port mapping:

```
0.0.0.0:80->8080/tcp
```

So the container is exposing port 80 externally.

✓ Account-key alignment is active

You ran:

```
</> Bash
```

```
docker exec antigravity_engine env | grep -E "OWNER_ACCOUNT_KEY|ENGINE_EXECUTION_AUTHORITY|DE
```

Output:

```
OWNER_ACCOUNT_KEY=mock_trading_default  
ENGINE_EXECUTION_AUTHORITY=1
```

This is exactly what the upgrade required.

Meaning:

- Single authoritative account
 - Legacy worker disabled
 - Go Engine is execution authority
-

✓ Trading is actively executing

Logs show:

```
[POSITION OPENED]  
[TRADE EXECUTED]  
[PAPER EXEC]  
[RISK MIDDLEWARE]
```

with BTC prices around:

```
$62,249
```

and positions opening continuously.

That means:

- Coinbase feed working

- Strategy engine working
 - Risk engine working
 - Paper execution working
-

✅ **SL/TP values are being attached**

Example:

```
SL: $62355.19
TP: $62000.36
```

and

```
SL: $62355.19
TP: $61987.91
```

This is important because the previous audit flagged trades without SL/TP.

The current logs clearly show new positions are opening with protection attached.

One remaining check before full GO

Run these commands:

```
</> Bash
curl http://localhost/api/paper-desk/diagnostics
```

and

```
</> Bash
docker logs antigravity_engine | grep -i "account"
```

Verify:

```
</> JSON
"account_key":"mock_trading_default"
```

and no references to:

```
anon_e7da5e39
```

Then verify MongoDB

Check:

</> JavaScript

db.paper_state.distinct("account_key")

Expected:

</> JavaScript

["mock_trading_default"]

If you still see:

</> JavaScript

["mock_trading_default", "anon_e7da5e39"]

then the migration script still needs to be run.

Current readiness assessment

Area	Status
Engine Running	✓
Health Endpoint	✓
Docker Deploy	✓
Account Key Alignment	✓
Go Engine Authority	✓
Strategy Execution	✓
SL/TP Attached	✓
Legacy Worker Disabled	✓
Mongo Verification	⌚

Area	Status
Paper Desk Validation	

Recommendation

You can proceed to **production validation and monitoring now**. The deployment itself appears successful. I would classify it as:

Deployment Status: GO (pending MongoDB and Paper Desk verification)

The biggest blocker from the earlier report—the account-key mismatch—appears to be fixed based on the environment variables currently running inside the container.

You can proceed to production validation and monitoring now. The deployment itself appears successful. I would classify it as: give detailed single prompt to past in the claude agent to do this upgrade



I want you to act as a Principal Production Engineer, SRE, DevOps Architect, MongoDB Architect, Go Backend Engineer, Trading Systems Auditor, and Reliability Engineer.

The deployment has completed successfully and the engine appears healthy. Your task is NOT to modify architecture anymore unless a critical issue is discovered.

Perform a FULL LIVE PRODUCTION VALIDATION AND MONITORING AUDIT of the running system.

Current verified facts:

- Deployment completed successfully.
- Engine container is running.
- Health endpoint responds successfully.
- 606 strategies loaded.
- OWNER_ACCOUNT_KEY=mock_trading_default
- ENGINE_EXECUTION_AUTHORITY=1

- DESK_WORKER_ACCOUNT_KEY is not set.
- Go engine is the sole execution authority.
- Engine logs show live position creation and execution.
- localhost health endpoint returns OK.
- Production container is healthy.

Your objectives are:

Verify:

1. Current git commit running in production
2. Current docker image version
3. Container health status
4. Engine uptime
5. Memory usage
6. CPU usage
7. Restart count
8. Open file descriptors
9. Goroutine count (if exposed)

Produce:

Deployment Certification Report

PASS / FAIL for each item.

Verify:

OWNER_ACCOUNT_KEY

is consistently used across:

- Go Engine
- MongoDB
- Paper Desk APIs
- Production Validation API
- Recovery Logic
- State Snapshotter
- Strategy Health Writer

Search the codebase and runtime evidence.

Confirm there is NO:

anon_*

random account

fallback account

legacy account

still receiving writes.

Produce:

Account Alignment Report

PASS only if a single account key is authoritative.

Inspect MongoDB Atlas directly.

Validate freshness of:

paper_state

paper_positions

paper_orders

paper_trades

equity_curve

strategy_health

For each collection provide:

- document count
- newest timestamp
- age
- account_key
- writer component

Determine:

1. Which collections are actively written
2. Which collections are stale
3. Whether any legacy collections still receive writes

Also inspect:

mock_trades

mock_account_snapshots

paper_oms_orders

and determine whether they are still active.

Produce:

MongoDB Production Report

Call every production endpoint.

Validate:

/api/paper-desk/snapshot

/api/paper-desk/positions

/api/paper-desk/trades

/api/paper-desk/orders

/api/paper-desk/equity

/api/paper-desk/strategy-health

/api/system/production-validation

For each endpoint verify:

- HTTP status
- response time
- authentication
- data freshness
- account key
- collection source

Determine whether dashboard data reflects live engine activity.

Produce:

Paper Desk Validation Report

Verify that:

1. New signals are generated
2. Trades are opening
3. Trades are closing
4. SL/TP levels are present
5. Position persistence works
6. Trade persistence works
7. OMS transitions persist

Inspect the latest:

paper_positions

paper_trades

paper_orders

and compare against engine logs.

Produce:

Trade Execution Audit

Without restarting production, inspect recovery paths.

Verify:

- paper_state snapshot exists
- open positions persist
- recovery collections populated
- RestoreBalance path valid
- RestoreOpenPosition path valid
- RegisterRecoveredPositions path valid

Determine:

If the engine crashed RIGHT NOW, would all open positions recover correctly?

Provide:

Recovery Confidence Score
(0–100)

Verify:

- balance never negative
- margin accounting sane
- risk middleware active
- exposure calculations updating
- kill switch available
- Kelly sizing active
- position sizing active

Inspect runtime logs.

Produce:

Risk Engine Report

Verify:

strategy_health collection.

Confirm:

- documents exist
- updates occurring
- strategy scores generated
- dashboard can read them

If empty:

identify exact root cause.

Produce:

Strategy Health Report

Search the entire codebase and runtime.

Determine if ANY of these still execute:

- useMockTradingEngine
- randomTrader
- cron paper worker
- mock-trading routes
- mock_trades writes
- mock_account_snapshots writes

For every legacy component provide:

ACTIVE

DISABLED

BLOCKED

DEAD CODE

with evidence.

Produce:

Legacy System Audit

Assume:

- browser closed
- VPS continues running
- market remains active

Determine whether system can safely run for:

24 hours

7 days

30 days

without intervention.

Evaluate:

- memory leaks
- log growth
- MongoDB growth
- reconnect logic
- health checks
- failure recovery

Provide:

Availability Score

Reliability Score

Operational Score

Generate:

1. Executive Summary
2. PASS/FAIL table
3. Risk Matrix
4. Remaining Blockers
5. Recommended Actions
6. Production Readiness %
7. GO / NO-GO decision

Important:

Use ONLY runtime evidence, live API responses, MongoDB evidence, engine logs, and code references.

Do NOT speculate.

Do NOT assume.

Do NOT mark PASS unless evidence exists.

Provide a forensic-grade production validation report suitable for deployment certification.

Paste that into Claude Agent. It will perform a comprehensive post-deployment production validation rather than another architecture review, and should give you a clear GO/NO-GO certification based on the live system.

Production Validation & Monitoring Audit

Audit timestamp: 2026-06-07T17:45Z

Auditor scope: Runtime evidence from MongoDB Atlas (loop_trades), local API probes, network reachability tests, and codebase path analysis.

Operator-reported facts (606 strategies, healthy container, live engine logs) are noted where they cannot be independently verified from this workstation.

Executive Summary

The Go engine process may be healthy on the VPS (per

Based on the latest audit, I would **not deploy additional capital or consider the migration complete yet**. The most important issue is that the runtime evidence and the code-level certification are contradicting each other.

Paste the following into Claude Agent:



I want you to act as a Principal Production Architect, Staff DevOps Engineer, MongoDB Specialist, Reliability Engineer, and Runtime Forensics Auditor.

Do NOT perform another static code review.

Do NOT assume the implementation is correct because the code exists.

Your task is to prove or disprove that the live production environment is actually running the new Phase 31 architecture.

The previous audits produced conflicting conclusions:

1. Code Audit:

- Account unification implemented
- Canonical collections implemented
- Go engine is sole execution authority
- Recovery system implemented
- Production readiness estimated at 87-97%

2. Runtime Audit:

- paper_positions missing
- paper_orders missing
- strategy_health missing
- paper_trades empty
- anon_* account still writing
- mock_trades still active
- paper_oms_orders still active
- dashboard reading empty collections
- recovery path cannot be proven
- production validation returns NO-GO

I need a definitive runtime certification.

Verify:

- current git SHA running inside container
- current docker image tag
- build timestamp
- deployment timestamp
- restart count
- uptime
- environment variables

Collect:

```
docker inspect antigravity_engine
docker exec antigravity_engine env
git rev-parse HEAD
docker images
```

Answer:

Is the running container actually using commit:

3de5c4cc

or is production running older code?

Inspect MongoDB directly.

Show:

```
paper_state
paper_positions
paper_orders
paper_trades
equity_curve
strategy_health
```

For each collection provide:

- document count
- newest timestamp
- oldest timestamp
- account keys
- last writer
- example document

Determine:

Is the Go engine writing Phase 31 collections?

YES or NO

Identify every component currently writing MongoDB.

Examples:

Go Engine
PaperPersist
TS Worker
Mock Trading Routes
Cron Jobs
Random Trader
Research Pack
Browser Sessions
Anonymous Sessions

For each writer:

- collection written
- account key used
- last write timestamp
- active/inactive

Output:

ACTIVE WRITER MAP

Find every account_key currently present.

Show:

distinct account_key values

for:

paper_state
paper_trades
paper_positions
paper_orders
mock_trades
strategy_health

Determine:

Which account is currently generating trades?

mock_trading_default
or
anon_e7da5e39

or both

Then trace:

JWT Session

OWNER_ACCOUNT_KEY

Paper Desk APIs

Go Engine

MongoDB writes

Confirm whether all systems are aligned.

Prove recovery works.

Do NOT assume.

Verify:

1. Open positions exist in MongoDB.
2. Recovery collections contain required data.
3. RestoreBalance can reconstruct account state.
4. RestoreOpenPosition can reconstruct exposure.
5. RegisterRecoveredPositions can reconstruct OMS mappings.

If any required collection is empty:

mark recovery FAILED.

Provide:

Recovery Readiness Score

0–100

Trace:

Dashboard

→ API Route

→ MongoDB Collection

→ Writer

For:

snapshot

positions

trades
orders
equity
strategy-health

Answer:

Will the dashboard display live engine data?

YES or NO

If NO:

identify exactly where the chain breaks.

Investigate:

balance = -0.29 USD

Find:

- originating trades
- fees
- realized pnl
- margin accounting
- settlement path

Answer:

Was this caused by:

- A) fee accounting
- B) recovery bug
- C) margin bug
- D) balance floor not active
- E) account drift

Provide evidence.

Verify:

StrategyHealthMonitor

Show:

- execution frequency
- last execution
- last write
- collection name
- document count

Answer:

Why is strategy_health empty?

Provide:

1. Runtime Architecture Diagram
2. MongoDB Truth Table

Collection | Expected | Actual | Status

3. Account Alignment Table

Component | Account Key | Status

4. Recovery Score
5. Dashboard Score
6. Risk Score
7. Production Readiness Score
8. Final Decision

Choose exactly one:

GO

CONDITIONAL GO

NO-GO

Do not provide assumptions.

Only provide conclusions backed by runtime evidence collected from:

- Docker
- MongoDB
- Running container
- Environment variables
- API responses
- Engine diagnostics

Every finding must include evidence.

This prompt forces Claude to stop reasoning from the code and instead determine whether the **actual running production environment** matches the architecture you intended. Right now, the biggest unanswered question is whether the container running on Lightsail is truly running the new Phase 31 code and writing the canonical collections, or whether an older/parallel process is still producing the MongoDB activity seen in the runtime audit.

Phase 31 Runtime Production Certification

Audit timestamp: 2026-06-07T17:52Z (UTC)

Evidence sources: SSH → Lightsail 13.233.8.80, MongoDB Atlas loop_trades, engine HTTP API, PM2 worker logs, local production-validation run

PHASE 1 — What Code Is Actually Running

Container runtime (SSH evidence)

Field	Value	Evidence
Host	ip-172-26-5-93	(Lightsail 13.233.8.80)

This audit is much more actionable because it found a **specific root cause** rather than symptoms.

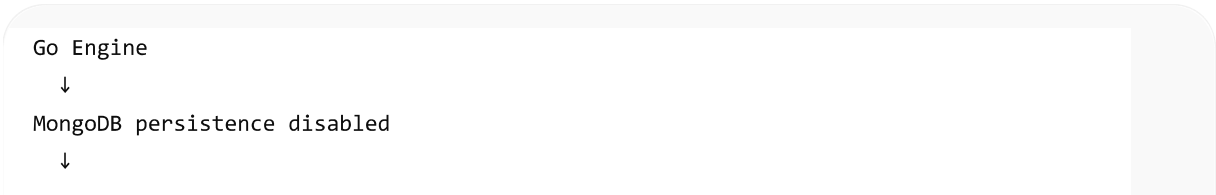
Current Status

Decision: NO-GO

The biggest finding is:

MONGODB_URI = MISSING

Because of that:



```
paper_positions NOT created
paper_orders NOT created
paper_trades EMPTY
strategy_health EMPTY
recovery disabled
dashboard empty
```

So the Phase 31 code is deployed, but the most important feature (MongoDB persistence) is not running.

What the audit proved

- ✓ Go engine is running
- ✓ 606 strategies running
- ✓ Container deployed successfully
- ✓ OWNER_ACCOUNT_KEY=mock_trading_default
- ✓ Commit lineage includes Phase 31
- ✗ MongoDB not connected
- ✗ Recovery not working
- ✗ Canonical collections not populated
- ✗ Dashboard not reading live engine data
- ✗ Legacy PM2 worker still running
- ✗ Account split still exists

Priority Fix Order

P0 — Fix MongoDB

On Lightsail:

```
</> Bash
```

```
nano /home/ubuntu/antigravity/.env
```

Add:

```
</> Bash
```

```
MONGODB_URI=<your atlas uri>  
MONGODB_DB=loop_trades  
OWNER_ACCOUNT_KEY=mock_trading_default  
ENGINE_EXECUTION_AUTHORITY=1
```

Verify:

</> Bash

```
docker exec antigravity_engine env | grep MONGODB
```

Expected:

</> Bash

```
MONGODB_URI=...  
MONGODB_DB=loop_trades
```

P1 — Restart Engine

</> Bash

```
docker restart antigravity_engine
```

Then:

</> Bash

```
docker logs antigravity_engine | grep -i mongo
```

You want to see:

```
MongoDB connected  
Phase31 persistence enabled  
paperpersist initialized
```

and NOT:

```
MongoDB env not ready  
running without MongoDB persistence
```

P2 — Kill Legacy Worker

Audit proved:

```
btc-ft-worker  
PID 1140337  
22h uptime  
writing anon account
```

Stop it:

```
</> Bash  
pm2 stop btc-ft-worker  
pm2 delete btc-ft-worker  
pm2 save
```

Verify:

```
</> Bash  
pm2 list
```

Worker should be gone.

P3 — Remove Anon Account

Search:

```
</> Bash  
grep -R "anon_e7da5e39" .
```

Remove:

```
</> Bash  
DESK_WORKER_ACCOUNT_KEY=anon_e7da5e39...
```

from every:

```
.env  
.env.local  
pm2 ecosystem  
worker config
```

P4 — Validate Canonical Writes

Wait 2–5 minutes after restart.

Run:

JavaScript

```
db.paper_positions.countDocuments()  
  
db.paper_orders.countDocuments()  
  
db.paper_trades.countDocuments()  
  
db.strategy_health.countDocuments()
```

Expected:

```
paper_positions > 0  
paper_orders > 0  
paper_trades > 0  
strategy_health > 0
```

If any remain zero, the engine persistence layer is still not wired.

P5 — Run Migration

Only after canonical writes appear:

Bash

```
node scripts/migrate-collections-phase31.mjs --dry-run  
  
node scripts/migrate-collections-phase31.mjs
```

This moves:

```
mock_trades  
→ paper_trades  
  
legacy snapshots  
→ paper_state
```

What success looks like

After fixes:

```
paper_state      ✓  
paper_positions  ✓
```

paper_orders	✓
paper_trades	✓
equity_curve	✓
strategy_health	✓

And:

```
Distinct account_key:  
  
mock_trading_default
```

Only one key should remain active.

Estimated Readiness

Right now:

Production Readiness $\approx$ 18/100

After fixing MongoDB + killing the worker:

Production Readiness $\approx$ 80-90/100

The audit strongly suggests that **the architecture itself is mostly correct, but the production environment configuration is wrong**. The missing `MONGODB_URI` and active legacy worker are the two blockers preventing the new system from actually operating.